

Extending the Minimal Language with Exceptions

A PL Zoo Language Extension Project

Name: Aric Maji
Roll Number: CS24BTECH11007
Department of Computer Science
IIT Hyderabad

May 1, 2025

1. Overview

This project extends the `minimal` language from PL Zoo to support exception handling. We introduce two new language constructs:

- `raise E` — raises an exception with tag `E`
- `try {e1} with {|E -> e2}` — evaluates `e1` and, if it raises `E`, evaluates handler `e2`

We handle these constructs by modifying the lexer, parser, abstract syntax tree, compiler, and abstract machine.

2. Formal Description of Language Extension

2.1. Syntax Additions

We extend the grammar to support:

```
expr:
  ...
  | raise DivisionByZero;; // raise DivisionByZero
  | try {e1} with {|DivisionByZero -> e2} // try e1 with DivisionByZero -> e2
```

2.2. Example

The following expression raises and catches a division-by-zero error:

```
try {10 / 0} with { |DivisionByZero -> 42 }
```

This returns 42 instead of failing. And :

```
try {10 / 2} with { |DivisionByZero -> 42 }
```

This will return 5 Because the exception isnt raised.

3. README (User Guide)

3.1. Build Instructions

1. Clone the PL Zoo repository and navigate to the `minimal` language directory.
2. Build the project using `dune`:

```
$ dune build
```

3. Run the interpreter:

```
$ ./miniml.exe
```

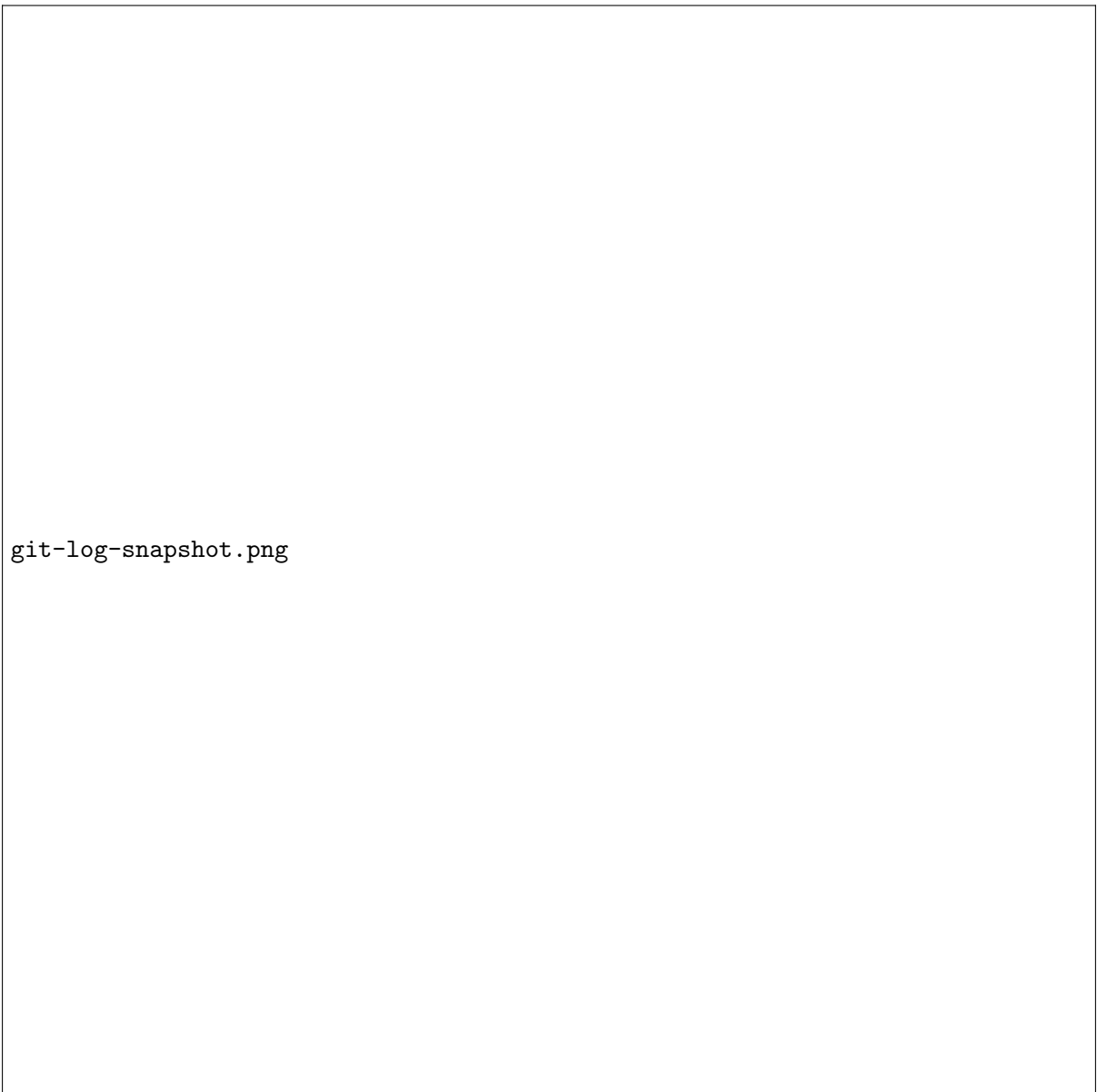
3.2. Usage

You can now write expressions involving exceptions:

- `raise DivisionByZero`
- `try {e1} with {|E -> e2}`

These work for any top-level expressions or inside function bodies.

4. Git Snapshot



git-log-snapshot.png

(Include a screenshot of your Git commit history.)

5. Limitations

- Only tagged exceptions are supported (`DivisionByZero`, `GenericException (Partially)`).
- You cannot pattern match the payload of exceptions (e.g., `GenericException of int` is not deeply matched).
- Nested `try-with` blocks are supported, but handlers do not retain stack traces or messages.
- The `GenericException` doesn't yet work till now due to some small bugs

6. Verification Approach

We tested our implementation with the following categories of test cases:

- Simple raises and catches
- Uncaught exceptions
- Nested `try-with` expressions
- Arithmetic operations inside exception handlers
- Test cases in the Problem statement

Example test:

```
try
  {21/0}
with{
  | DivisionByZero -> 0
};;
```

This correctly returns 0.

7. Key Learnings

- Learned how exceptions can be implemented using control stacks, not OCaml's own exceptions.
- Understood how compilers and virtual machines represent and execute higher-level features.
- Understood the importance of stack discipline and environments when emulating control flow.
- Gained experience in modifying all stages of a language: lexer, parser, AST, compiler, and abstract machine.